Using AI Tools in Mechatronics Projects

A practical introduction

Parsa Ghasemi

Autonomous Vehicle Laboratory

Overview

- What tools are available
- How to use them effectively
- Where they commonly make mistakes
- Live coding: building a motor controller
- Two demos combining AI with hardware

What This Talk Covers

This is a short practical guide. The goal is that you can open one of these tools tonight and use it for your own work.

- We will not cover the underlying theory
- We will not compare every tool on the market
- We will focus on workflow and common pitfalls

Tools

Two Tools We Will Use

Gemini CLI

- Free
- 1,000 requests per day
- Google account sign-in
- Open source

Claude Code

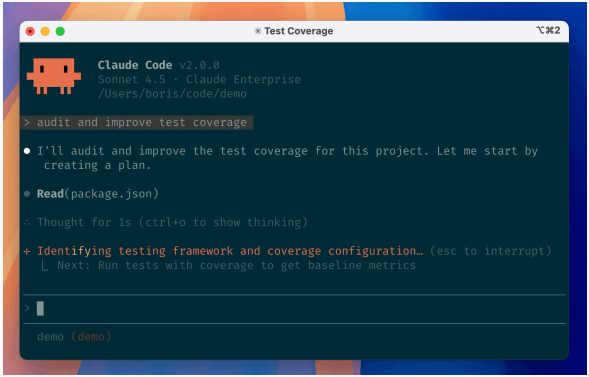
- Paid ($20/month)
- Generally stronger reasoning
- Good for embedded work
- Handles larger projects

Either tool works for everything shown today. The free option is a reasonable starting point.

Gemini CLI

A simple REPL in your terminal. You type what you want; it reads, writes, and edits files in your project.

Claude Code



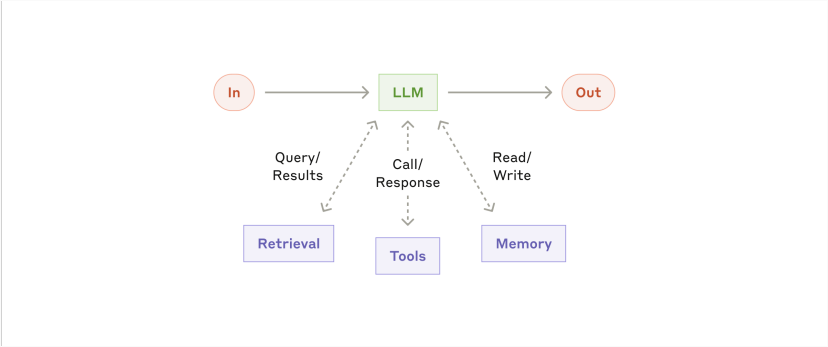
The same idea from Anthropic. Often better at longer tasks and embedded code, but requires a paid subscription.

Why a Terminal Interface

Terminal-based AI tools differ from chat windows in a few practical ways:

- They can read the files in your project directory
- They can edit files directly, without copy-paste
- They can run shell commands, including build and flash tools
- They keep context across a full working session

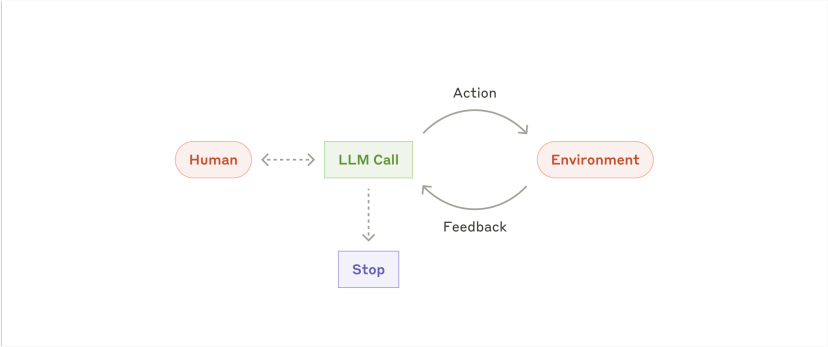
What an Agentic CLI Actually Does



The underlying language model is augmented with access to **tools** (shell, file edits, compilers), **retrieval** (your project files), and **memory** (session context). The CLI is the harness that wires these together.

Source: Anthropic, "Building Effective Agents"

The Agentic Loop



You state a goal; the model takes an action (edits a file, runs a command); it observes the feedback (error, output); and it iterates until the task is done or it asks you for help.

Source: Anthropic, "Building Effective Agents"

Installation (macOS / Linux)

Prerequisite: Node.js 18 or later, from `nodejs.org`

```
# Gemini CLI
npm install -g @google/gemini-cli
gemini

# Claude Code
npm install -g @anthropic-ai/claude-code
claude
```

On first run, sign in with a Google account (Gemini) or Claude account.

Windows: Consider Using WSL

WSL (Windows Subsystem for Linux) provides a Linux environment inside Windows, without dual boot or a virtual machine.

It is worth using because:

- Most AI tooling is tested on Linux first
- Most tutorials online assume a Linux shell
- Path and permission issues are rarer

It is not required. Native Windows works too.

Installing WSL

1. Open PowerShell as Administrator
2. Run:

```
wsl --install
```

3. Restart the computer
4. On restart, Ubuntu opens. Set a username and password.

Installing the CLI Inside WSL

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

sudo npm install -g @google/gemini-cli
gemini
```

The VS Code WSL extension is useful if you prefer a graphical editor while working inside Linux.

Writing Effective Prompts

A Useful Prompt Structure

Prompts tend to work better when they include, in some form:

1. A brief role or framing
2. The specific context of your setup
3. A clear task
4. Relevant constraints
5. Some form of verification

These do not need to be labeled as such. The point is to give the tool enough information to answer precisely.

An Example Prompt

I am using a Teensy 4.1 with a DC motor driver on pins 3 and 4, and a quadrature encoder on pins 2 and 7.

Please write an Arduino sketch that spins the motor one full rotation forward and then stops.

Use the Encoder library. The encoder produces 5760 counts per revolution.

Briefly explain each part of the code, and list the wiring.

Vague Prompts vs Specific Prompts

Vague

"Write me motor code."

Without context, the tool has to guess: which board, which driver, which pins, which libraries.

Specific

"I have a Teensy 4.1 with a motor driver on pins 3 and 4. Please write a sketch that spins forward for 2 seconds, stops, then reverses."

Now the reply is targeted and usually runnable.

Providing Context

It is often helpful to share:

- Relevant sections of a datasheet
- Your wiring, in a few lines of text
- A photo of the circuit
- The existing code you are editing
- The exact error message you encountered

A useful rule of thumb: if you would need to explain it to a classmate for them to help, the tool likely needs it too.

Common Mistakes

Where These Tools Often Go Wrong

Even capable models make confident mistakes. Common ones include:

- Incorrect pin numbers
- Unit errors (radians vs degrees, RPM vs rad/s)
- Function names that do not exist in the library
- Code written for a different board
- Part numbers that are not real
- Links that do not resolve

A Small Example

Prompt: *"Write Arduino code to spin a motor at 50% speed."*

```
int motorPin = 9;

void setup() {
  pinMode(motorPin, OUTPUT);
}

void loop() {
  analogWrite(motorPin, 50);  // 50% speed, right?
}
```

The Issue

`analogWrite` accepts values from **0 to 255**, not 0 to 100.

A value of 50 corresponds to roughly 20% duty cycle, not 50%. The code compiles, the motor spins, and the behavior looks reasonable. It is simply not what was asked for.

Ways to Catch These

A short checklist that helps:

- Read each line before running it
- Compile early; syntax errors surface fake APIs quickly
- Test the simple cases (known input, known expected output)
- Confirm the wiring matches what the code expects
- Ask the tool to review its own code for bugs

Over time, you learn the kinds of mistakes a given tool tends to make.

Live Coding

Why a Live Demo

Everything so far has been description. The purpose of coding live is:

- To show the actual pace and rhythm of the workflow
- To show how mistakes appear and get corrected in practice
- To give you something to compare against when you try it yourself
- To connect what follows: both demos reuse this firmware

Building a Motor Controller

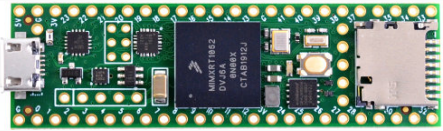
In the next few minutes, we will build a position-controlled motor from an empty folder, using only the terminal.

The plan:

1. Spin the motor
2. Read the encoder
3. Accept setpoints over serial
4. Add a PID controller

Hardware Setup

- Teensy 4.1, connected by USB
- Motor driver: RPWM on pin 3, LPWM on pin 4
- Encoder: channel A on pin 2, channel B on pin 7
- Gear motor, 5760 counts per revolution
- Bench power supply for the motor



Teensy 4.1

Live Demo

Live coding

A backup sketch is prepared in case of issues.

Brief Recap

- We started from an empty folder
- The CLI wrote the sketch, compiled it, and flashed the board
- A few mistakes appeared and were corrected with additional prompts
- The result is a working position controller

The workflow is the main point: describe, review, test, iterate.

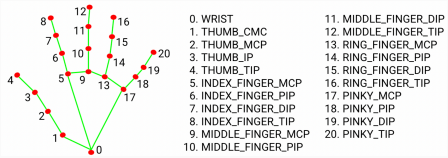
Two Further Demos

Demo 1 — Vision to Motor

The webcam tracks a hand; the motor follows its position.

- MediaPipe Hands (Google, free)
- A short Python script to read the webcam and send setpoints
- The same PID firmware on the Teensy

Approximate build time: a few hours.



MediaPipe hand landmarks

Demo 1 — Data Flow

Perception from the model; control from the microcontroller. Python and Teensy talk over USB serial.

Demo 1: Live

Vision to Motor

Demo 2 — Voice to Motor

A spoken command is transcribed locally and mapped to a motor action.

- OpenAI Whisper, running on the laptop (no internet needed)
- A short parser that recognizes a handful of commands
- The same Teensy firmware

Approximate build time: a couple of hours.

Demo 2 — Data Flow

Everything runs locally. No internet required after the first model download.

Demo 2: Live

Voice to Motor

A volunteer is welcome.

Suggestions

A Way to Get Started

1. Install Gemini CLI or Claude Code
2. Ask it to walk you through a datasheet page
3. Have it draft your next sketch, and read through carefully
4. Paste a real bug and ask for a diagnosis
5. Keep notes on the mistakes you notice

A Few Things Worth Remembering

- These tools are useful; they are not a substitute for understanding
- Most of the productivity gain is on boilerplate and lookup
- Verification remains the engineer's responsibility
- Clear prompts generally matter more than picking the newest model
- The free options are sufficient to begin

Thank You

Questions or discussion welcome.

Parsa Ghasemi
Autonomous Vehicle Laboratory
github.com/Paarseus

References i

AI Tools

- Gemini CLI — `github.com/google-gemini/gemini-cli`
- Claude Code — `claude.com/claude-code`
- Claude web chat — `claude.ai`
- Cursor — `cursor.com`
- GitHub Copilot (free for students) — `education.github.com/pack`

Demo Libraries

- MediaPipe Hands — `google.github.io/mediapipe`
- OpenAI Whisper — `github.com/openai/whisper`
- Arduino CLI — `arduino.github.io/arduino-cli`

References ii

- Encoder library — github.com/PaulStoffregen/Encoder
- PID_v1 library — github.com/br3ttb/Arduino-PID-Library

Embedded Toolchain

- Teensyduino — pjrc.com/teensy/teensyduino.html
- WSL — learn.microsoft.com/windows/wsl
- usbipd-win — github.com/dorssel/usbipd-win